

Yadage to Snakemake: translation reference

This maps the original Yadage workflow (`workflow/*.yaml` , driven by `reana.yaml`) to the Snakemake port (`Snakefile` , driven by `reana-snakemake.yaml`). Two tables: general concepts first, then the concrete stage-by-stage mapping for this specific analysis.

Concept mapping

Yadage	Snakemake	Notes
<code>stages:</code> (named list of steps)	<code>rule <name>:</code> blocks	Same unit of work, different declaration style
<code>scheduler_type: singlestep-stage</code>	a rule with no wildcards	Runs exactly once
<code>scheduler_type: multistep-stage</code> + <code>scatter:</code>	a rule with wildcards, fanned out via <code>expand()</code> in whatever rule consumes its output	One job per scattered value either way
<code>scatter: {method: zip, parameters: [a, b]}</code>	multiple wildcards in one output path, e.g. <code>{mc}_{shapevar}</code>	Index-wise pairing becomes wildcard combinations Snakemake resolves itself
<code>batchsize: N</code> on a merge stage	not reproduced	See "Where the structure changed" below
<code>dependencies: [stageA, stageB]</code>	nothing explicit; inferred from <code>input:</code> / <code>output:</code> paths matching	Snakemake builds the DAG from file paths, not a declared list
<code>parameters: {stages: X, output: Y}</code>	<code>input:</code> referencing another rule's <code>output:</code> path, or plain Python values from <code>PROFILES</code> /wildcards	Same idea, resolved differently
<code>workflow: {\$ref: sub.yaml}</code> (nested sub-workflow)	flattened into more rules in the same <code>Snakefile</code>	No nesting; Snakemake's <code>include:</code> exists but wasn't needed here
<code>step: {\$ref: steps.yaml#/name}</code> (shared packtivity template)	the <code>shell_cmd()</code> helper, or duplicated <code>shell:</code> text where the command differs per rule	Sometimes a shared function, sometimes just repetition
<code>process.process_type: interpolated-script-cmd</code> + <code>script: </code> with <code>{param}</code> placeholders	<code>shell:</code> string with <code>{wildcards.x}</code> / <code>{params.x}</code> / <code>{input}</code> / <code>{output}</code>	Direct equivalent, same interpolation idea
<code>environment.environment_type: docker-encapsulated</code> + <code>image</code> + <code>imagetag</code>	<code>container: IMG_MAIN</code> (a <code>docker://image:tag</code> string)	Direct equivalent

<code>publisher.publisher_type: frompar-pub</code> + <code>outputmap</code>	<code>output: declaration</code>	The declared output path is the published file; no separate publish step
<code>publisher.publisher_type: interpolated-pub</code> + <code>glob: true</code> (makews's <code>{prefix}*combined*model.root</code>)	a fixed, explicit output path	Our <code>makews.py</code> call controls the exact filename, so there's nothing to glob for
top-level input files (<code>inputsm.yml</code> , <code>inputsig.yml</code> , <code>allmc_input.yml</code>) feeding Yadage's implicit <code>init</code> stage	<code>PROFILES dict + config.get("profile", "full")</code>	Same role: initial parameters for the whole run

Where the structure changed

The Yadage sub-workflows (`workflow_mc.yml` , `workflow_data.yml` , `workflow_sig.yml`) select on partially-merged batches and merge again afterward, controlled by `batchsize` . For example `workflow_mc.yml` 's `merge` stage has `batchsize: 2` over 4 generated chunks, producing 2 partial merges; `select_signal` then runs once per partial merge, and `select_signal_merge` combines the selected outputs back into one file.

The Snakemake port collapses this: `merge_generated` always combines every batch for a sample into a single file right after generation, and every `select_*` / `histogram_*` rule downstream runs once on that complete file. Fewer rules, fewer jobs, same final numbers, since `hadd` and the selection cuts don't care where the batch boundaries were. This is also why several Yadage stages below map to "not needed" rather than a named rule.

Stage-by-stage mapping

databkgmc.yml (top level)

Yadage stage	Snakemake rule
<code>all_bkg_mc</code>	flattened, see <code>wflow_all_mc.yml</code> / <code>workflow_mc.yml</code> below
<code>data</code>	flattened, see <code>workflow_data.yml</code> below
<code>signal</code>	flattened, see <code>workflow_sig.yml</code> below
<code>merge</code> (step <code>merge_root_allpars</code>)	<code>merge_all</code>
<code>makews</code>	<code>makews</code>
<code>plot</code>	<code>plot</code>
<code>hepdata</code>	<code>hepdata</code>

workflow_data.yml

Yadage stage	Snakemake rule
read	generate (sample=data)
merge	merge_generated (sample=data)
select_signal	data_select (region=signal)
select_signal_merge	not needed (merge_generated already merged everything)
select_control	data_select (region=control)
select_control_merge	not needed
select_signal_hist	data_hist_signal
select_control_hist	data_hist_control
mergeall	data_mergeall

workflow_sig.yml

Yadage stage	Snakemake rule
read	generate (sample=sig)
merge	merge_generated (sample=sig)
select	sig_select
select_merge	not needed
select_hist	sig_hist
hist_merge	folded into sig_hist 's output (only ever one histogram file)

wflow_all_mc.yml + workflow_mc.yml (instantiated once per mc1/mc2 via the {mc} wildcard)

Yadage stage	Snakemake rule
run_mc (scatters workflow_mc.yml over mc1/mc2)	replaced by the {mc} wildcard on every mc_* rule
workflow_mc.yml: read	generate (sample=mc1 / mc2)
merge	merge_generated (sample=mc1 / mc2)
select_signal (weight variations)	mc_select_weights
select_signal_merge	not needed
select_signal_hist	mc_hist_weights
mergeweights	folded into mc_hist_weights 's single output
select_signal_shapevars (scatters workflow_select_shape.yml)	replaced by the {shapevar} wildcard on mc_select_shape / mc_hist_shape

<code>mergeshapes</code>	folded into <code>mc_hist_shape</code> 's single output
<code>mergeallvars</code> (in both files)	<code>mc_mergeallvars</code>

workflow_select_shape.yml (instantiated per {mc} x {shapevar})

Yadage stage	Snakemake rule
<code>select</code>	<code>mc_select_shape</code>
<code>merge</code>	not needed
<code>hist</code>	<code>mc_hist_shape</code>

steps.yml (packtivity templates, reused across the stages above)

Yadage packtivity	Snakemake equivalent
<code>generate</code>	<code>generate</code> 's <code>shell:</code> , calling <code>code/generantuple.py</code>
<code>select</code>	<code>data_select</code> / <code>sig_select</code> 's <code>shell:</code> , calling <code>code/select.py</code> with <code>nominal</code> only
<code>select_mc</code>	<code>mc_select_weights</code> / <code>mc_select_shape</code> 's <code>shell:</code> , calling <code>code/select.py</code> with variations
<code>histogram</code>	<code>data_hist_*</code> / <code>sig_hist</code> / <code>mc_hist_weights</code> 's <code>shell:</code> , calling <code>code/histogram.py</code>
<code>histogram_shape</code>	<code>mc_hist_shape</code> 's <code>shell:</code> , calling <code>code/histogram.py</code> with a name template
<code>merge_root</code>	every plain <code>hadd</code> -based rule (<code>merge_generated</code> , <code>data_mergeall</code> , <code>mc_mergeallvars</code> , ...)
<code>merge_root_allpars</code>	<code>merge_all</code>
<code>makews</code>	<code>makews</code> 's <code>shell:</code> , calling <code>code/makews.py</code>
<code>plot</code>	<code>plot</code> 's <code>shell:</code> , the <code>hfquickplot</code> calls
<code>hepdata</code>	<code>hepdata</code> 's <code>shell:</code> , calling <code>code/hepdata_export.py</code> + <code>zip</code>

Input parameter files

Yadage file	Snakemake equivalent
<code>allmc_input.yml</code> (mcname, mcweight, nevents for mc1/mc2)	<code>PROFILES[...]["nevents"]["mc1"/"mc2"]</code> , <code>PROFILES[...]["mc_weights"]</code>
<code>inputsig.yml</code> (signal nevents, mcweight)	<code>PROFILES[...]["nevents"]["sig"]</code> , <code>PROFILES[...]["sig_weight"]</code>
<code>inputsm.yml</code> (data nevents)	<code>PROFILES[...]["nevents"]["data"]</code>

The two profiles in `Snakefile` (`full` , `test`) roughly correspond to the numbers in these three files versus a scaled-down local set.